

[ASRJC/2025/Q4]

4 Name your Jupyter Notebook as:

TASK4_<your name>_<centre number>_<index number>.ipynb

ASRTech Solutions is a technology company that provides subscription-based services to its users. The company currently has user information in a text file `users.txt` containing user IDs, names, emails and salted hashed credit card numbers.

While actual credit card processing is handled securely through external payment services, ASRTech maintains hashed versions of credit card numbers within its own system. The hashing process uses a fixed salt string which prevents rainbow table attacks by hackers. Storing these salted hashed credit card numbers allows the company to perform secure lookups and comparisons. For example, the company can check if a card has been used before without risking exposure of sensitive data.

The company currently also maintains a separate text file `credits.txt` that tracks all credit transactions. Each transaction record contains the user ID, date and time of transaction, transaction type, and amount. These credits can be topped up or deducted.

For each of the sub-tasks 4.1 to 4.4, add a comment statement at the beginning of the code using the symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 4.1
Program code
```

Output:

Task 4.1

Write Python program code (with the use of SQL commands) to:

- create a database `USER_DATA` with the following tables:
 - `User`: storing user information, using field names that match those in the text file and using a suitable field as the primary key,
 - `Credit`: storing credit transaction information, using field names that match those in the text file, with a composite key consisting of the user's ID and the datetime of transaction, ensuring referential integrity between the two tables.(You should ensure that the fields are of appropriate data types.)
- read the data from the text files and insert them into the database.

Run your program to transfer the data to the database and display the number of inserted users and transaction records. [6]

Task 4.2

The Luhn algorithm is a checksum formula used to validate credit card numbers and detect errors. How it works is as follows:

1. Starting from the rightmost digit (excluding the final check digit), double the value of every second digit.
2. If doubling results in a 2-digit number, add the digits together instead.
3. Sum all the digits.
4. If the total sum is divisible by 10, it passes the Luhn validation check, and the card number is considered valid.

[ASRJC/2025/Q4 Cont']

Consider the credit card number **4417123456789113**.

Step 1: **8, 4, 2, 7, 2, 2, 6, 4, 10, 6, 14, 8, 18, 1, 2, 3**

Step 3: $8 + 4 + 2 + 7 + 2 + 2 + 6 + 4 + (1 + 0) + 6 + (1 + 4) + 8 + (1 + 8) + 1 + 2 + 3 = 70$

Step 4: 70 divided by 10 produces a remainder of 0, so card number is considered **valid**.

The function `validate_card(card_number)` takes `card_number` as a parameter, which is a string of digits, and implements the Luhn algorithm to return a boolean indicating whether the card number is valid.

Test your function with the following test cases:

- 4532840913491054 (16 digits)
- 5555734352999785 (16 digits)
- 378282904122908 (15 digits)
- 4111025640404402 (16 digits)
- 5105207030955616 (16 digits)

[6]

Task 4.3

Write a function `check_user(user_id)` that runs an SQL query to determine whether the given user ID is present in the `User` table of the database. The function should return `True` if the user ID is found and `False` otherwise.

Test with suitable test cases.

[3]

Task 4.4

The user management system should allow users to update their personal credit card information.

Note that the database should only store salted hashed credit card numbers.

Use the following fixed salt: "@SR_SUP3R_S3CURE_S4LT!".

To create the salted hash, concatenate the salt with the card number (salt + card number), then apply SHA-3 (a strong cryptographic hash function) to the combined string.

Refer to the sample code provided in `sample_code.py` for SHA-3 implementation.

Write program code to declare a function `update_card(user_id, new_num)` that:

- checks whether the user ID exists in the database
- checks whether the new credit card number is valid using the Luhn algorithm
- generates a salted hash of the new credit card number using SHA-3
- updates the credit card hash in the database for the specified user ID.

The function should return an appropriate success or error message.

Display the output of the following:

```
update_card("swimmerz", "30569309025904")
update_card("koh-world", "1122334455667788")
```

[6]

Save your Jupyter Notebook for Task 4.

[ASRJC/2025/Q4 Cont']

Task 4.5

ASRTech Solutions requires a web application that allows staff to search for users and view their transaction history.

Write a Python program and the necessary files to create a web application that:

- provides a search interface where the admin can enter a user's name
- performs name searches in the database
- shows a table of all credit transactions made by the user within the past 90 days (with reference to the server's current system date)

Below are some additional requirements:

- the table should have 3 columns: Date and Time, Transaction Type, Transaction Amount
- transactions within the timeframe should be displayed from the latest to the oldest
- the background colour of the row in the table should be #58D68D if the transaction amount is positive, and #FFC300 if negative.

Save your Python program with any additional files/subfolders in a folder named:

TASK_4_5_<your_name>_<centre_number>_<index_number>

Below is a mock-up of how the web application interface might look:

ASRTech Solutions

View Transaction Records

Search by name:

90-day transaction history for Emmanuel

Date and Time	Transaction Type	Transaction Amount
2025-08-12 09:31:27	top-up	925.0
2025-08-10 14:47:29	top-up	650.0
2025-06-27 16:45:31	top-up	950.0
2025-06-21 13:19:38	deduction	-188.8

Run the web application and test with a search of the user "Caleb Tay".

Save the web page output as:

TASK_4_5_<your_name>_<centre_number>_<index_number>.html

[11]